

Numeric Width

I. Introduction

This paper proposes a number of template definitions for manipulating the width of numeric types. It's primary aim is to make it easier to widen numeric types in the case that arithmetic operations would otherwise cause overflow. It's specific application is the `fixed_point<>` class template detailed in [P0037R2](#).

II. Impact on the Standard

The following definitions are proposed for addition to header, `<type_traits>`:

- class templates, `width<>` and `set_width<>`;
- helper alias template, `set_width_t<>` and
- helper variable template, `width_v<>`.

III. Prior Art

[Fixed width integer types](#) provide width-specific aliases to built-in integer types. However, they do not provide a generic way to specify a type from a given width value. [Boost.Integer](#), on the other hand, provides this functionality through [Integer Type Selection](#).

Both sets of definitions have variants that allow the user to choose between the fastest and most compact alternatives. However, resultant types are confined to built-in integers; there is no way to specify user-defined types.

In contrast, this proposal aims specifically to facilitate generic widening of integer types - including user-defined types. Most of the functionality was present in [revision 1](#) of fixed-point arithmetic proposal, P0037, as the `resize<>` class template. A current implementation can be found in the [reference implementation](#) of P0037.

IV. Motivation

Results of arithmetic operations - such as addition and multiplication - have greater range than their operands. This means that overflow errors are possible in common situations. For instance:

1. When the result is stored in a type that is the same as the operands:

```
// range of a*b is UCHAR_MAX*UCHAR_MAX but range of return value is UCHAR_MAX
uint8_t multiply(uint8_t a, uint8_t b) {
    return a*b;
}
```
2. When the result is stored in an automatically deduced type but the range is too great:

```
// range of a*b is UINT_MAX*UINT_MAX but range of return value is UINT_MAX
auto multiply(unsigned a, unsigned b) {
    return a * b;
}
```

It is left up to the user to ensure that capacity meets demand:

```
auto multiply(uint32_t a, uint32_t b) {
    using result_type = uint64_t;
    return result_type{a} * result_type{b};
}
```

Currently, the standard does not support a generic implementation of the above:

```
template<class Operand>
auto multiply(Operand a, Operand b) {
    using result_type = ???;
    return result_type{a} * result_type{b};
}
```

`Boost.Integer` provides a generic API which works for built-in integers of a known signedness:

```
template<class Operand>
auto multiply(Operand a, Operand b)
{
    constexpr auto operand_width = sizeof(Operand)*CHAR_BIT*2;
    using result_type = typename boost::uint_t<operand_width>::fast;
    return result_type{a}*result_type{b};
}
```

A more broadly-applicable solution should take the type of `Operand` into account and not just its capacity.

Use Case: Fixed-point Arithmetic

One benefactor of such a generic solution is class template, `fixed_point<>`, from [P0037R2](#):

```
template<class Rep, int Exponent>
class fixed_point;
```

The default type for `Rep` is `int`. This means that `fixed_point<int>` uses `int` as its underlying storage. However, any suitably integer-like type can be specified for `Rep`.

There are certain situations where use of `fixed_point<>` requires generic widening of type. These mostly relate to arithmetic operations. For instance, the binary multiply operator widens the intermediate type in order to store the extra digits involved. A more advanced application of `fixed_point<>` is to produce auto-widening types - such as those introduced by Lawrence Crowl in [P0106](#). Such types have operators which return results of larger types than their operands, effectively eliminating overflow in many cases.

V. Design

The proposed library additions are intended to manipulate the width of integer types at compile time.

Determining the Width of an Integer

The width of a given type can be determined using definitions, `width<>` and `width_v<>`

```
template<class T>
struct width;
```

```
template<class T>
constexpr unsigned width_v = width<T>::value;
```

such that the following assertions always hold:

```
static_assert(width<uint16_t>::value == 16, "the width of uint16_t is exactly 16 bits");
static_assert(width<long long>::value >= 64, "long long has a width of at least 64 bits");
static_assert(width<long>::value >= width<short>::value, "short is not longer than long");
static_assert(width<wchar_t>::value >= width<char>::value, "a wide character is at least as wide as a character");
```

Specifying the Width of an Integer

Definitions, `set_width<>` and `set_width_t<>`, take an input type and a width in bits and yield an equivalent type which has at least that many bits.

```
template<class Archetype, unsigned MinNumBits>
struct set_width;
```

```
template<class Archetype, unsigned MinNumBits>
using set_width_t = typename set_width<Archetype, MinNumBits>::type;
```

They are similar in usage to `make_signed/make_signed_t` and `make_unsigned/make_unsigned_t` which also take one type and produce another (possibly different) type.

The following assertions either fail to compile or hold true:

```
static_assert(is_same<set_width_t<signed, 8>, int8_t>::value, "int8_t is a signed 8-bit integer");
static_assert(is_same<set_width_t<unsigned, 32>, uint32_t>::value, "uint32_t is an unsigned 32-bit integer");
static_assert(is_same<set_width_t<uint64_t, 16>, uint16_t>::value, "a 64-bit unsigned integer was narrowed to 16-bits");
static_assert(is_same<set_width_t<char, 64>, int64_t>::value || is_same<set_width_t<char, 64>, uint64_t>::value, "char may or may not be signed so the result may be uint64_t or int64_t");
static_assert(is_same<set_width_t<int, 10>, int16_t>::value, "result must be at least 10 bits wide");
```

Families and Archetypes

A family is a set of types which differ in width alone. The set of *signed* built-in integers is one family. The set of *unsigned* built-in integers is another. No type can belong to more than one family.

The Archetype parameter of `set_width` identifies a family by any one of its members. The result of `set_width` is defined as the most narrow family member with at least `MinNumBits` of width. If `width_v<T>` equals `N`, then `set_width_t<T, N>` is guaranteed to be `T`.

Specializations of `width` and `set_width` are provided for built-in types. However, custom types may be specialized by the user.

VI. Technical Specification

Header *<type_traits>* Synopsis

```
namespace std {
    template<class T>
    struct width;

    template<class Archetype, unsigned MinNumBits>
    struct set_width;

    template<class T>
    constexpr unsigned width_v = width<T>::value;

    template<class Archetype, unsigned MinNumBits>
    using set_width_t = typename set_width<Archetype, MinNumBits>::type;
}
```

VII. Outstanding Issues

API Details

The name `set_width` is almost certainly the wrong choice. The use of the word 'set' makes it sound like a run-time operation. Possible alternatives include `make_width`, `of_width` and `make_integer`.

The order of the two template parameters for `set_width` and `set_width_t` is by no means cast in stone. It may be that defaulting Archetype to `int` is a good idea in which case, swapping the order would certainly be on the cards.

Fast and *least* variants to `set_width` and `set_width_t` would probably make valuable additions.

The choice of `unsigned` as the type used to express width is by no means final. Possible alternatives include `size_t` and `int` but `unsigned` happens to have worked best in practice thus far.

Non-Integer Types

The possibility of specializing for the floating-point family has not been ruled out. It is not clear what width means in the context of floating-point types. Possibly it means the mantissa or the entire content of a type, in which case `width_v<float>` is either 24 or 32.

Extended / Future Standard Integer Types

Some compilers define a non-standard 128-bit integer type. Types of this width may enter the standard in the future. For this reason, it is left unspecified whether these types join in the families of built-in integers. Whether a library implementation chooses for `set_width_t<signed, 128>` to cause a compiler error or yield `__int128` on supported compilers is a matter for consideration.

Acknowledgements

Special thanks to Brent Friedman, Marco Foco, Arthur O'Dwyer and Michael Wong for their valuable input.